

Compte Rendu de Projet

Jeu de Combat Pokémon en Java

Auteur	Hugo Madoumier Marius Migard
Niveau	Première année
Langage	Java
Technologies	Java, SQLite, JDBC

1. Présentation du projet

Ce projet est un jeu de combat Pokémon entièrement jouable en console, développé en Java dans le cadre du cours de première année. L'objectif était de concevoir un programme orienté objet complet, depuis la modélisation des données jusqu'à la persistance en base de données.

Le joueur compose une équipe de 3 Pokémon parmi les 151 de la première génération, puis affronte une intelligence artificielle en combat au tour par tour, avec les règles de types fidèles au jeu original.

1.1 Fonctionnalités principales

- Sélection de l'équipe parmi 151 Pokémon avec affichage paginé (20 par page)
- Combat au tour par tour : attaquer ou changer de Pokémon
- Table des types complète de la 1ère génération (15 types, multiplicateurs $\times 2$ / $\times 1$ / $\times 0,5$ / $\times 0$)
- Deux modes de jeu : match unique et tournoi (3 combats consécutifs)
- Quatre niveaux de difficulté pour l'IA
- Tableau des scores mis en base de données SQLite
- Gestion des scores : consultation, modification de nom, suppression

2. Modes de jeu et niveaux de difficulté

2.1 Modes de jeu

Le jeu propose deux modes distincts :

- Match unique : un seul combat contre une IA. Le score est enregistré à l'issue du combat.
- Tournoi : trois combats consécutifs contre trois IA différentes. L'équipe du joueur est soignée entre chaque combat. Un unique score global est enregistré à la fin.

2.2 Niveaux de difficulté

Quatre niveaux ont été implémentés, chacun enrichissant la logique du précédent :

Niveau	Comportement de l'IA
Facile	Choisit ses attaques entièrement au hasard
Moyen	Sélectionne l'attaque dont le score (puissance $\times$ multiplicateur de type) est le plus élevé
Difficile	Comme Moyen, et change stratégiquement de Pokémon si le matchup de type est défavorable
Impossible	Comme Difficile, avec un bonus de +30 % en attaque et en défense appliqué dès le départ

3. Architecture du code

Le projet suit une architecture orientée objet claire, avec une séparation des responsabilités entre les différentes classes.

3.1 Description des classes

Classe	Rôle
Pokemon	Modélise un Pokémon : nom, type, PV, attaque, défense, liste d'attaques (max 4)
Attaque	Représente une attaque avec son nom, son type et sa puissance
Equipe	Contient jusqu'à 3 Pokémon et gère le Pokémon actuellement en combat
Type (enum)	Enumère les 15 types de la 1ère génération
TableTypes	Calcule le multiplicateur de dégâts selon le type de l'attaque et celui du défenseur
NiveauDifficulte (enum)	Enumère les 4 niveaux de difficulté de l'IA
Joueur	Gère les interactions clavier du joueur humain (choix d'attaque, changement de Pokémon)
JoueurIA	Hérite de Joueur et surcharge les méthodes de décision selon le niveau de difficulté
Combat	Orchestre la boucle de jeu tour par tour et calcule les dégâts
BaseDeDonnees	Gère la connexion SQLite et toutes les opérations CRUD sur les scores
Main	Point d'entrée : menu principal, navigation, composition des équipes

3.2 Héritage et polymorphisme

Le point central de la conception objet est la relation entre Joueur et JoueurIA. La classe JoueurIA hérite de Joueur et surcharge deux méthodes :

- `choisirAttaque()` : l'IA sélectionne automatiquement son attaque selon son niveau, sans interaction clavier.
- `choisirPokemonSuivant()` : l'IA envoie le premier Pokémon non KO disponible.

La classe Combat ne distingue pas si `joueur2` est humain ou IA grâce au polymorphisme : elle appelle simplement `choisirAttaque()` sur chaque joueur. La logique spécifique à l'IA (`doitChanger`, `changerVersMeilleur`) est appelée séparément via un cast `instanceof JoueurIA`.

3.3 Calcul des dégâts

La formule de dégâts est inspirée de la formule officielle des jeux Pokémon, simplifiée pour la console :

dégâts = ((puissance × attaque / défense) / 5 + 2) × multiplicateur_type

Un minimum de 1 point de dégâts est garanti si le type n'est pas immunisé. Le multiplicateur est récupéré depuis la classe TableTypes.

4. Persistance des données

Les scores sont enregistrés dans une base de données SQLite via JDBC (driver sqlite-jdbc). Le choix de SQLite s'impose pour un projet console : léger, sans serveur, et le fichier .db est créé automatiquement au premier lancement.

4.1 Opérations disponibles

- Enregistrement automatique d'un score après chaque combat (nom du joueur, résultat, nombre de tours, date)
- Affichage du tableau des scores trié par résultat et date
- Modification du nom d'un joueur dans un score existant
- Suppression d'un score individuel
- Suppression de la totalité de l'historique

Les données des Pokémon (nom, type, stats) sont quant à elles chargées depuis le fichier texte pokemon_data.txt au démarrage, via un parsing ligne par ligne avec séparateur point-virgule.

5. Difficultés rencontrées

5.1 La logique de l'IA

La principale difficulté a été de concevoir une IA qui soit réellement progressive. En Difficile et Impossible, l'IA doit décider avant chaque tour si elle change de Pokémon ou si elle attaque. Cela implique de comparer les multiplicateurs de types de tous ses Pokémon disponibles par rapport au Pokémon adverse, sans que ce traitement ne ralentisse la boucle de jeu.

La solution retenue est d'appeler `doitChanger()` avant chaque tour dans `Combat`, et de ne faire le changement que si un meilleur Pokémon existe réellement.

5.2 La gestion du changement de Pokémon

Changer de Pokémon en cours de tour implique que le joueur « perd » son tour d'attaque, mais que l'adversaire attaque quand même. Gérer correctement l'ordre des actions et les cas limites (Pokémon KO juste après un changement, équipe entière KO) a nécessité plusieurs itérations.

5.3 La sélection paginée des Pokémon

Afficher 151 Pokémon en console sans que ce soit illisible a nécessité un système de pagination (20 Pokémon par page). La navigation par commandes textuelles (D / G / numéro) devait aussi empêcher de choisir deux fois le même Pokémon.

6. Bilan et perspectives

6.1 Ce qui a été réalisé

Le projet atteint l'ensemble des objectifs fixés : un jeu jouable, une IA avec plusieurs niveaux de comportement, une persistance des scores, et un code structuré en classes aux responsabilités clairement définies. La séparation entre la logique de jeu (`Combat`, `Joueur`) et les données (`Pokemon`, `Equipe`, `BaseDeDonnees`) facilite la lisibilité et la maintenance.

6.2 Pistes d'amélioration

- Ajout d'une interface graphique (JavaFX ou Swing) pour remplacer la console
- Attaques avec effets de statut : paralysie, sommeil, empoisonnement, etc.
- Mode multijoueur local (deux joueurs humains sur la même machine)
- Système d'expérience et de montée de niveau pour les Pokémon
- Sauvegarde et rechargement d'une partie en cours

Conclusion

Ce projet a permis de mettre en pratique les concepts fondamentaux de la programmation orientée objet en Java : héritage, polymorphisme, encapsulation et séparation des responsabilités. L'intégration de SQLite via JDBC a également apporté une dimension concrète de persistance des données.

Le résultat est un jeu fonctionnel et complet, dont la structure en classes pourrait être réutilisée comme base pour une version avec interface graphique.